CulturaGenAI System v3.2

CulturaGenAI v3.2 — System Deployment, Security and Maintenance Manual

Release date: 8 November 2026
Technical baseline: 39 source-level automated checks passed on the v3.1 code that is retained in this release; private Supabase acceptance tests remain pending.

Release architecture and separation of duties

```
PUBLIC GITHUB REPOSITORY (frontend ZIP only)
  Landing + Academy synthetic demo + independent AdaptEngine + Pilot UI
  Open poster / 76-slide deck / sample certificate
                    |
                    | Public publishable key only, after authorisation
                    v
PRIVATE SUPABASE PROJECT AND PRIVATE GITHUB REPOSITORY
  Auth + institution memberships + PostgreSQL RLS
  Private 30-practice / 18-exam bank + server-side marking
  Educator reviews + audit trail + completion records
  Optional NADI read-only LLM agent; server-held API keys
```

Never upload the complete release ZIP, master-source folder, or private-backend ZIP to a public repository. Code publication is not the same as deploying files: sensitive assessment bank content could be exposed directly in the GitHub source browser even if GitHub Pages publishes only its static folder.

Release bundle contents

- `CulturaGenAI_v3_2_Public_GitHub_Frontend.zip`: public pages, synthetic learner assets, GitHub Actions CI, poster and teaching files.
- `CulturaGenAI_v3_2_PRIVATE_Supabase_Backend.zip`: private grading bank, migrations, Edge Function and privileged workflow source.
- `competition-materials/`: editable research poster, A1 PDF and PNG, 76-slide deck and sample certificate.
- `manuals/`: user manual, system manual, security evidence notes, canonical module register and competition claims ledger.

v3.2 curriculum and release controls

- Treat `curriculum/CURRICULUM_REGISTER.md` as the canonical six-module register for titles, outcomes and final-exam mapping. Do not independently rename or reorder modules in a poster, lesson slide, public application, private assessment bank or completion record.
- `web/academy/src/data.js` and `web/pilot/app.js` use the same official English M01–M06 titles; the final-exam bank maps `F01–F18` to the six IDs without exposing answer keys publicly.
- To update a module: revise the register, translations, learning notes, practice bank, private final exam bank and assessment rubric as one controlled change; version the curriculum, obtain academic approval, rerun source tests, and record migration impacts on past learner evidence.
- Only publish the **public GitHub ZIP**; the **private backend ZIP includes exam answers** and must remain private. Never place secret values in browser files.
- In v3.2, changes to application logic are **presentation-only**: complete module titles and an informational module list on the pilot completion record. Production readiness remains conditional.

Deployment model — two repositories

Repo A (public): GitHub Pages source = **frontend ZIP ONLY**. Contains synthetic Academy, AdaptEngine, secure pilot UI (but NO server grading bank) and GitHub Actions workflow. Never add `.env`, service keys or backend files.

Repo B (private): Supabase source = **backend ZIP ONLY**. Contains SQL migration, exam question bank, authenticated Edge Function and DB access tests. Keep the repo private because **server-side answer keys are confidential even if no API secrets are present**.

This separation is essential. A GitHub Pages workflow that publishes only `web/` does **not** stop visitors from reading non-published files in a public GitHub source repository.

1. Publish frontend in GitHub (quickest demonstration)

1. Create a new GitHub repository named `culturagenai-academy`; it may be public **only if it contains frontend ZIP files exclusively**.
2. Upload the **contents** of `CulturaGenAI_Public_Frontend_GitHub.zip` to the repository root (not the parent ZIP).
3. Push to branch `main`.
4. Under **Settings → Pages**, set **Build and deployment → Source: GitHub Actions**.
5. The workflow runs the public demo tests, checks the public files, and uploads the static site. Visit `https://<owner>.github.io/<repo>/` (replace owner/repo).
6. Verify the landing page, `/academy/`, `/adaptengine/` and `/pilot/` with a phone and desktop.
7. Without Supabase configuration, the secure pilot intentionally displays a configuration notice; this is **expected**.

2. Prepare a private Supabase project

1. Create a new Supabase project. Enable suitable Auth settings (prefer invite-only; require email verification and MFA for administrator accounts).
2. Create a **private** GitHub repository and upload **only** the Backend ZIP contents, keeping `supabase/` at repo root.
3. Install the Supabase CLI locally and sign in; use the official instructions for your OS.
4. From your private backend repo directory:

```
supabase login
supabase link --project-ref YOUR_PROJECT_REF
supabase db push
supabase functions deploy pilot
```

1. Run DB RLS tests **in the local Supabase CLI/Docker test environment**:

```
supabase start
supabase test db
```

1. The initial `supabase/migrations/202611080001_initial.sql` migration enables RLS on all app tables, blocks browser writes, and defines a server-only final exam transaction and daily agent quota.
2. In Supabase's Authentication settings, configure allowed redirect URLs and disable unrestricted user signup before a real pilot (unless explicitly needed).
3. Configure secure TLS and a recognised organisation domain; use Supabase-managed backups or scheduled database backups with restore testing.

2.1 Optional GitHub Actions deployment from the PRIVATE repo

The private backend ZIP includes `.github/workflows/deploy.yml`. Configure GitHub repository **Actions secrets**: `SUPABASE_ACCESS_TOKEN`, `SUPABASE_DB_PASSWORD`, and `SUPABASE_PROJECT_REF`. Create a GitHub environment named `production` with required human approvers. Select **Actions → Manually deploy private CulturaGenAI backend → Run workflow**, enter `DEPLOY`, and approve the deployment. The workflow links the project, applies approved migrations and deploys the protected Edge Function. Never store `OPENAI_API_KEY` as a public repo variable; set it directly through the Supabase secret manager.

2.2 Configure institution accounts (trusted administrator only)

First invite users through the Supabase Auth dashboard. Confirm each user has actually completed registration. Then use the SQL Editor with administrator privileges:

```
-- Example only — substitute genuine, authorised institutional details.
insert into public.institutions (name,slug)
values ('Example Pilot Institution','example-pilot')
returning id;
```

```
-- Look up the correct enrolled user's auth.users ID; do NOT grant role using
       browser metadata.
select id,email from auth.users where email='learner@example.edu';

-- Replace both UUID placeholders with results verified by your administrator.
insert into public.memberships (institution_id,user_id,role)
values ('INSTITUTION_UUID','USER_UUID','learner');
```

Repeat with `educator` roles for actual academic reviewers. Only authorised administrators should create or change memberships. Do not grant arbitrary users `admin` to speed up testing.

3. Connect public frontend to Supabase

In **public frontend repo** `pilot/config.js`, edit only:

```
window.CULTURA_CONFIG = Object.freeze({
  supabaseUrl: 'https://YOUR_PROJECT_REF.supabase.co',
  publishableKey: 'sb_publishable_YOUR_PUBLIC_KEY'
});
```

The publishable key is intended to be public. The **service-role/secret key is never allowed in the browser**. Re-push to `main` and wait for GitHub Pages deployment.

The pilot's HTML Content Security Policy permits `https://*.supabase.co`; if you use a custom Supabase domain, change the allowlist after review. For a production custom hosting solution, configure real HTTP response headers and a tested CSP, not only a meta tag.

4. Activate optional NADI real LLM (private backend only)

Without an AI secret, `pilot` provides an **offline planner** that uses authenticated learner progress and course titles. This is still functional.

To enable a live LLM with **bounded read-only tool calls**:

```
supabase secrets set OPENAI_API_KEY=YOUR_OPENAI_KEY
supabase secrets set OPENAI_MODEL=gpt-4.1-mini
supabase functions deploy pilot
```

Keep keys in the Supabase secrets manager. Never commit them, paste them in `pilot/config.js`, or expose them through client logs. Confirm your selected model is accessible and understand provider data handling, minors policy and billing before activation.

NADI's limits are: at most **10 requests per authenticated learner/day**, two read-only tool calls, bounded model output, explicit learner confirmation for proposed navigation, and no privileged database write tools. Audit events exclude raw prompts. Rate limits on other API actions and IP-based abuse controls still require additional operational configuration.

5. Security acceptance checklist (complete before real learners)

- [ ] Invite-only identity provision and role approval; enable MFA for staff.
- [ ] Two institutions + multiple test accounts: demonstrate that cross-tenant access is rejected for SELECT and write APIs.
- [ ] `supabase test db` passes the pgTAP grants/RLS suite and manual cross-tenant probes.
- [ ] Confirm public GitHub source and Pages artifacts contain **no** `question-bank.ts`, secret keys, database exports or learner data.
- [ ] Verify grading remains server-side by editing answers and role fields in browser developer tools.
- [ ] Confirm educator cannot approve own evidence, and cannot read another institution's submissions.
- [ ] Check expired, duplicated and resumed exam sessions; test two-attempt cap.
- [ ] Verify completion issuance requires a passed final exam and six approved submissions.
- [ ] Perform consent, privacy and ethics review for actual learners (particularly minors).
- [ ] Establish access requests, retention/deletion, reporting and incident-response procedures.
- [ ] Apply database backups and test **restoration**; test uptime alerts and error notifications.
- [ ] Perform independent security review, dependency/SAST scanning and concurrent-user/load tests before institutional rollout.
- [ ] Calibrate/adapt BKT and validate course items, translations, cultural examples and assessment criteria.

6. Maintenance runbook

Weekly: review Supabase security/auth alerts, failed function requests, account provisioning, unexpected agent request volume, capacity and provider spend. Avoid logging student prompts or personal records.

Monthly: patch dependencies and provider SDK versions, run all automated and database security tests, review authorised reviewers, audit exam/approval events and check backup

jobs. Rotate secrets when indicated by incidents or policy, not by copying them into the repository.

At each release: use a branch and peer review; verify migration in staging, run CI, perform live login/quiz/review regression tests, then deploy backend first and frontend after. Keep rollback/migration plan, incident contact and release change log.

Incident response: immediately disable affected API/agent or revoke compromised token/secret, identify affected tenants without leaking unrelated data, preserve minimal relevant audit metadata, notify your institutional data officer, restore from known-good backups where needed, and document remediation.

Scaling: measure P95 latency, database query plans and index performance, simultaneous authenticated learners, server function quotas, LLM costs, educator review time and multi-campus tenant isolation. Do not claim a capacity rating without a repeatable load test.

7. Known technical limitations

- The static Academy and AdaptEngine are **synthetic demo experiences**, not real accounts.
- The secure pilot supports authenticated records but **has not undergone external penetration testing, production load testing or legal/privacy acceptance**.
- The secure bank has 30 practice items and 18 final exam items; item difficulty, discrimination and BKT assumptions are **not validated**.
- Printable completion records are not accredited certificates and do not verify legal identity or external public QR validation.
- Browser session tokens use sessionStorage rather than HttpOnly cookies (a practical static-hosting compromise); a dedicated production domain with a secure BFF/cookie session is preferable for high-assurance deployments.
- Usage limits are implemented for live NADI only. Broader per-IP and write-action rate-limiting/WAF are a **future security hardening requirement**.
- Current SQL DB tests verify RLS/grant presence, but full two-tenant integration and penetration tests are still required.

Documentation and academic release checklist

Before an award submission or production pilot, reconfirm the six official module titles in `CURRICULUM_REGISTER.md`, the deck, the final bank and the certificate wording. Rerun all source checks (`npm run check`) and inspect real browser workflows. Record any score, learning-impact or learner-count claim with dated, consented source evidence. Keep an approved data protection impact assessment, supported institutional operating policy and incident contact list.